Week 13
Advanced Optimization for LLMs

Hwanjun Song
KAIST, Dept. of ISE

Today's Contents

- Transition in Optimization with Foundation Models <<
- Pre-training – unsupervised learning
- Post-training – supervised/reinforcement learning

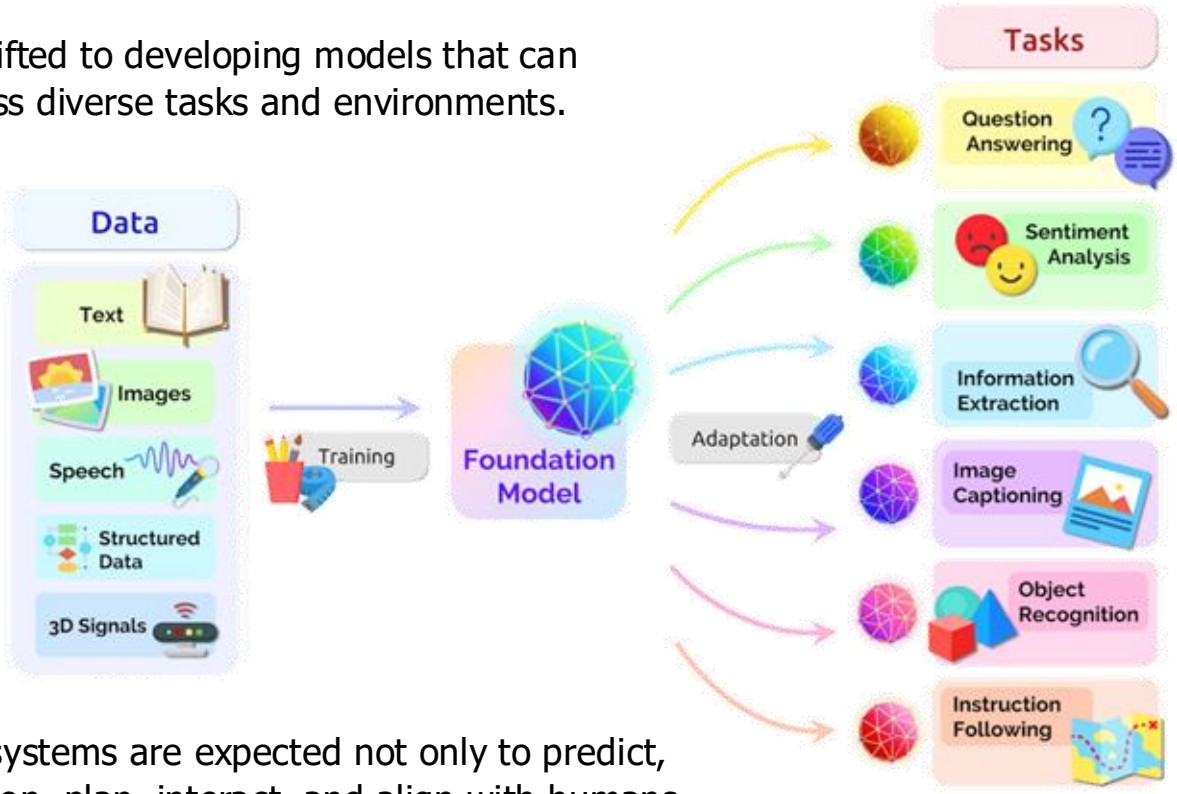
Historical Transition in Optimization (1/3)

- Classical DL Optimization
  - Model: Small CNNs, RNNs, or Transformers
  - Objective: A single target task such as regression, classification, detection, etc
  - Data: Supervised Data with {x, y}
  - Optimization: SGD, Adam, AdamW, or other variants
- Scaling Focus for A Better Model
  - Increasing supervised data set size
  - Increasing the number of trainable parameters
  - Optimization was mainly about minimizing task loss under supervised learning
- This paradigm works well when the goal is to train a specialized model for a clearly defined task

Historical Transition in Optimization (2/3)

- Demand Shift in Modern AI
  - Past: Image Classification, Object Detection, Speech Recognition, etc
  - Current: A **general-purpose** intelligence what we call "foundation model"

(1) The goal shifted to developing models that can generalize across diverse tasks and environments.



(2) Modern AI systems are expected not only to predict, but also to reason, plan, interact, and align with humans.

Historical Transition in Optimization (3/3)

- From mathematical perspective:
  - Past: single supervised objective – discriminative models

$$\theta^* = \arg\min_{\theta} \mathcal{L}_{task}(f_\theta(x), y)$$

  - Current in LLMs: we now consider mixture of objectives:
    - (1) Generating natural and coherent language – generative models
    - (2) Following human instructions
    - (3) Aligning with human preferences (e.g., safety, factuality)
    - (4) Too use and interaction for agentic AI

$$\mathcal{L}_{total} = \mathcal{L}_{pretrain} + \lambda_1 \mathcal{L}_{instruction} + \lambda_2 \mathcal{L}_{alignment} + \lambda_3 \mathcal{L}_{agentic}$$

$\mathcal{L}_{pretrain}$: next-token prediction from large-scale pretraining
$\mathcal{L}_{instruction}$: instruction-following behavior
$\mathcal{L}_{alignment}$: alignment with human preferences and safety
$\mathcal{L}_{agentic}$: reasoning, planning, tool use, and adaptive interaction

KAIST

Why Not Train Everything at Once?

- Satisfying all objective in parallel is too challenging:
  - Different objectives have conflicting behaviors
  - Different data distribution for each objective
    - Not possible to a perfect large data to cover all perspectives
  - Different optimization characteristic
    - Some for supervised learning; Some for Reinforcement Learning, Some for Continual learning, etc.
- A single optimization stage cannot effectively balance all objectives simultaneously
- Rather than learning everything at once, modern LLMs progressively acquire capabilities stage by stage.

Modern LLM Training Pipeline

Pre-training -> Instruction Tuning -> Alignment -> Agentic Adaptation

Post-training

- "Pre-training" to learn general language patterns by mimicking large-scale text distributions – *Unsupervised learning*
- "Instruction tuning" to follow human instructions and task formats – *Supervised learning*
- "Alignment tuning" to optimize helpfulness, safety, and human preferences – *Reinforcement learning*
- "Agentic Adaptation" to improve reasoning, planning, tool use, and adaptive *interaction – Supervised learning*

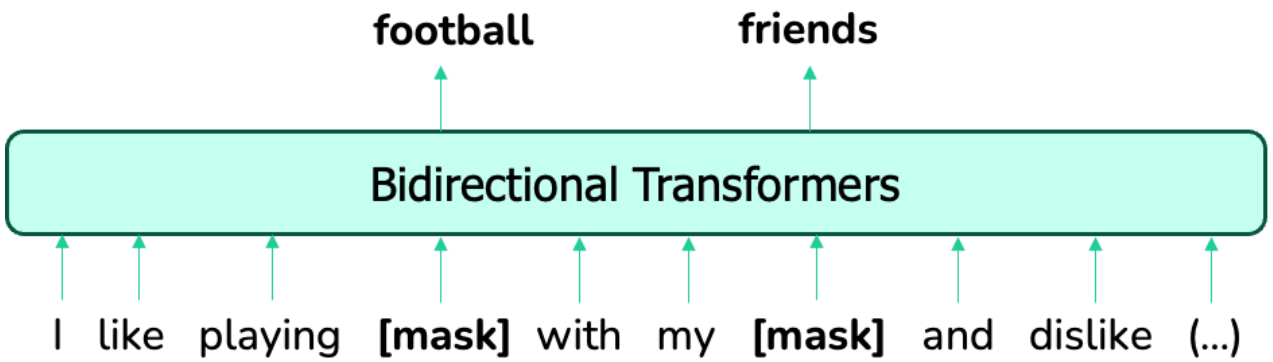
Today's Contents

- Transition in Optimization with Foundation Models
- Pre-training – unsupervised learning <<
- Post-training – supervised/reinforcement learning

Pre-training Stage in the Past (w. BERT)

- The most fundamental task that learns linguistic patterns and basic structures, enabling efficient performance across a wide range of NLP tasks
  - Widely used even before the emergence of LLMs for small LMs
- What kinds of tasks help shape this capability in the model?
  - **Next token prediction in BERT [2018, NAACL]**
  - Unsupervised learning by masking a few tokens in the input context

Two tasks: (1) Masked language modeling; (2) Next sentence prediction



Pre-training Stage in the Past (w. BERT)

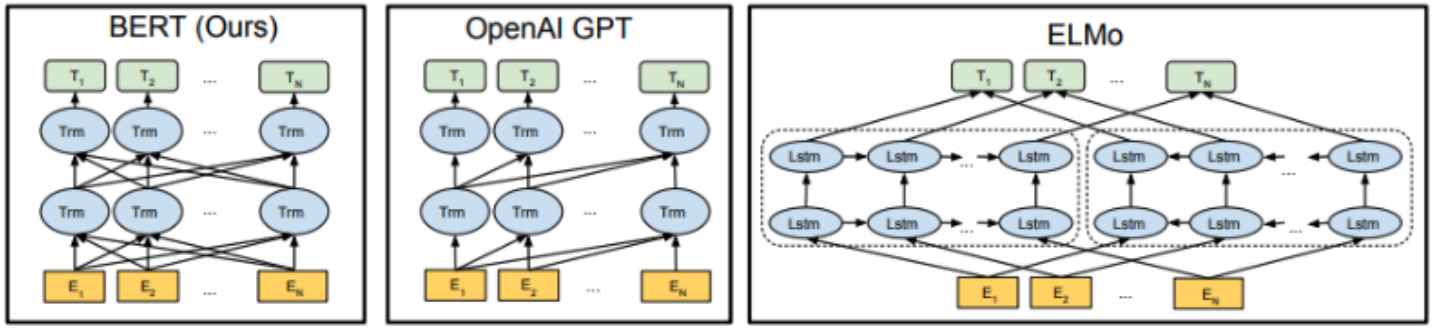
- Task 1: Masked Language Modeling (MLM)
  - Goal: Learn diverse and robust language patterns from large-scale text data
  - Method: Randomly mask 15% of input tokens during training
    - 80% of the time, replace with [MASK]
    - 10% of the time, replace with a random word
    - 10% of the time, keep the original token unchanged
- Task 2: Next Sentence Prediction (NSP)
  - Goal: MLM mainly learns representation within a single sentence, while NSP was introduced to help the model understand relationships between two sentences
  - Method :

Positive Pair	**Negative Pair**
[CLS] the man went to the [MASK] store [SEP] he bought a gallon [MASK] milk [SEP]	[CLS] the man [MASK] to the store [SEP] penguins [MASK] flightless birds [SEP]
Label = IsNext	Label = NotNext

Pretraining Stage: Other Variants

- There were other pretraining methods as well
  - All of them have the same objective for pretraining
  - 2018 Feb, ELMo (Embeddings from Language Models)
  - 2018 Jun: OpenAI GPT (Generative Pre-trained Transformers)
  - 2019 Feb: OpenAI GPT-2

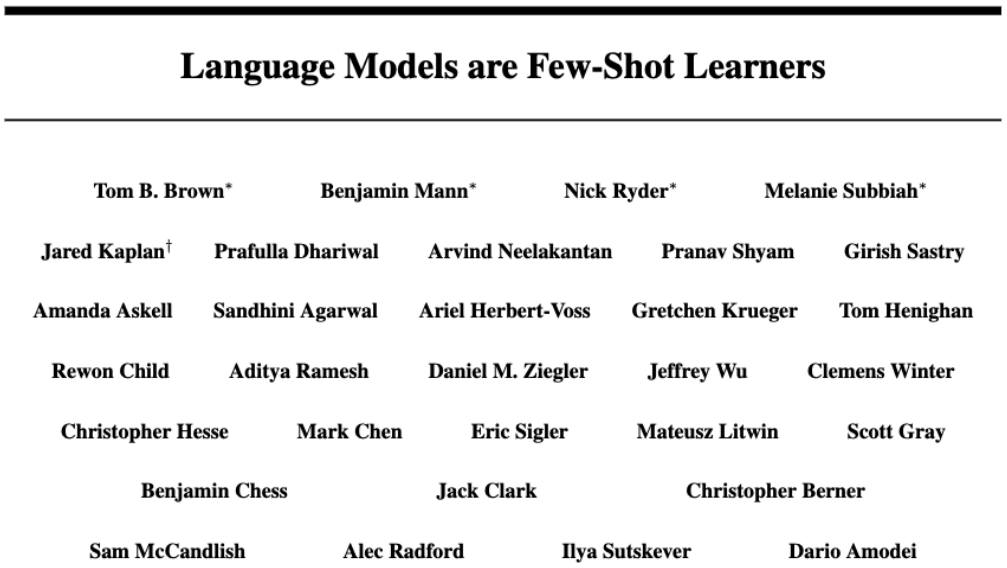




(Architecture)	Transformer encoder	Transformer decoder	Bidirectional LSTM
(Pre-trained LM)	Masked LM & NSP	Uni-directional LM	Bidirectional LM

GPT-3: Pre-training for ChatGPT

- Before LLM emerged, BERT is commonly used since the architectures are Encoder-Decoder seq-seq models.
  - Not suitable for decoder-only models which most LLMs have adopted
- LLMs are now widely adopting GPT-3 based pre-training specialized for Decoder-only models



Language Models are Few-Shot Learners

Tom B. Brown* Benjamin Mann* Nick Ryder* Melanie Subbiah*

Jared Kaplan† Prafulla Dhariwal Arvind Neelakantan Pranav Shyam Girish Sastry

Amanda Askell Sandhini Agarwal Ariel Herbert-Voss Gretchen Krueger Tom Henighan

Rewon Child Aditya Ramesh Daniel M. Ziegler Jeffrey Wu Clemens Winter

Christopher Hesse Mark Chen Eric Sigler Mateusz Litwin Scott Gray

Benjamin Chess Jack Clark Christopher Berner

Sam McCandlish Alec Radford Ilya Sutskever Dario Amodei

NeurIPS 2020, OpenAI

GPT-3: Pretraining for ChatGPT

- Objective of GPT-3 Pre-training
  - The task is simple, just predicting the next token given the previous context
  - So it is called a **causal language modeling** that maximizes the likelihood of the next token conditioned on all previous tokens:

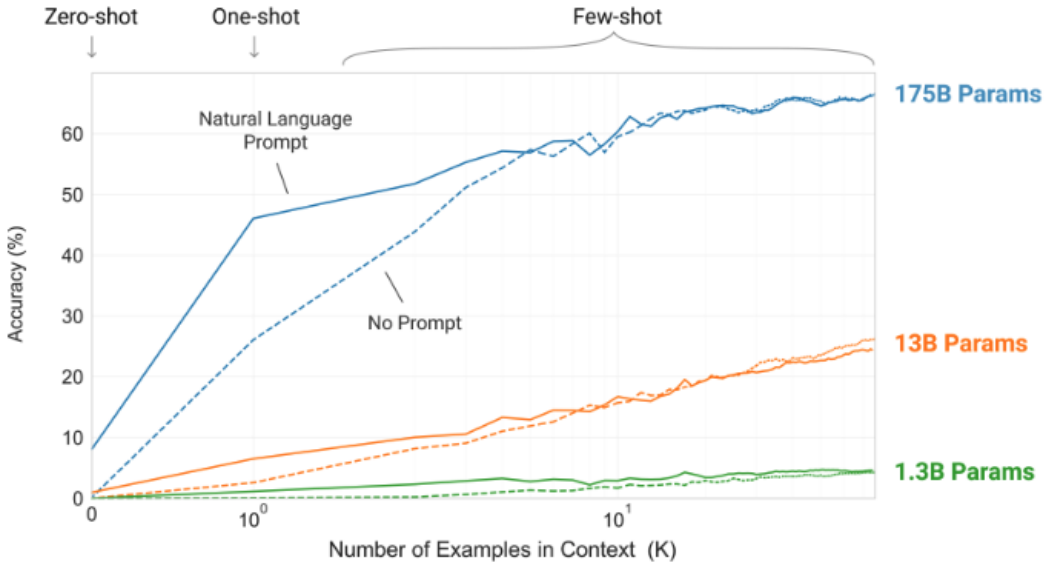
$$\text{Minimize} \quad \mathcal{L} = \sum_{t=1}^{T} \log P(x_t \mid x_{<t})$$

  - Specialized for autoregressive decoding of generating the text from left to right
    - Input: The cat is on the / Output: mat

- Why not Masked Autoencoding?
  - It directly matches the generation process used at inference time – mismatch
  - It enables autoregressive text generation even for long-form outputs
  - Decoder-only scaling empirically showed stronger zero- / few-shot learning abilities at large scale <- What does this mean?

Zero-shot / Few-shot Learning after GPT-3

- Before LLMs, a LMs require an additional fine-tuning on the model with a designated training set for a target task
- However, the zero-shot / few-shot ability is the capability of solving a new task without further training, but:
  - Zero-shot learning: No example given for a new task
  - Few-shot learning: A few examples given for a new task



(1) Model scaling
(2) Pre-training

Limitation of Pretraining

- Still, pre-training has fundamental limitations for practical usages
  - First, next token prediction is just a casual inference, so it is lacking in terms of understanding the user's dynamic / diverse prompts
    - E.g., {"Summarize this article in one sentence." "Can you briefly explain the main point?" "Give me a TL;DR." "What are the key takeaways from this document?" "I'm busy. Just tell me the important part."}
  - Second, pre-training also lacks explicit control over output formats and response styles. In prace, users often require outputs in highly specific formats such as:
    - E.g., {"Answer with only Yes or No." "Return the result in JSON format." "Translate it into a three-line poem." "Provide the output as a markdown table." "Explain it at an elementary school level."}

This is why we need additional post-training after pre-training.

Today's Contents

- Transition in Optimization with Foundation Models
- Pre-training – unsupervised learning
- Post-training – supervised/reinforcement learning <<

Instruction Fine-tuning

- A stage of training LLMs to be a model that is able to understand users' instruction and intent very well

Scaling Instruction-Finetuned Language Models

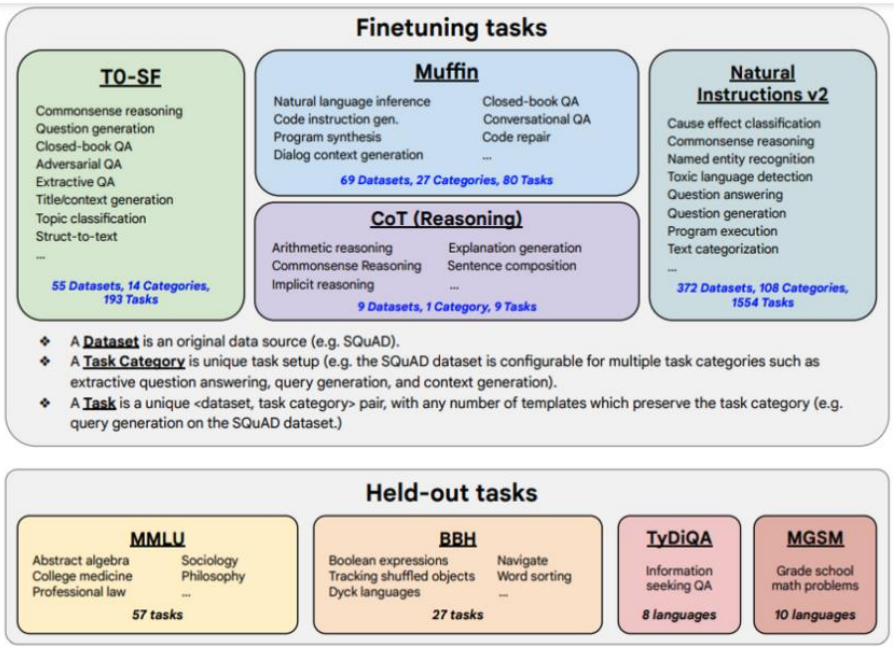
Hyung Won Chung* Le Hou* Shayne Longpre* Barret Zoph† Yi Tay†
William Fedus† Yunxuan Li Xuezhi Wang Mostafa Dehghani Siddhartha Brahma
Albert Webson Shixiang Shane Gu Zhuyun Dai Mirac Suzgun Xinyun Chen
Aakanksha Chowdhery Alex Castro-Ros Marie Pellat Kevin Robinson
Dasha Valter Sharan Narang Gaurav Mishra Adams Yu Vincent Zhao
Yanping Huang Andrew Dai Hongkun Yu Slav Petrov Ed H. Chi
Jeff Dean Jacob Devlin Adam Roberts Denny Zhou Quoc V. Le
Jason Wei*

Google

Released on arXiv 2022
(Published at JMLR in 2024)

Instruction Fine-tuning

- A stage of training LLMs to be a model that is able to understand users' instruction and intent very well
- The approach is simple:
  - First, data construction for instruction fine-tuning:
    - Input: {instruction, context} / Output : Answer



In data, it is required to collect as many instruction as possible

- 473 datasets
- 146 task categories
- 1836 total tasks

Covers Non-CoT tasks and CoT tasks

KAIST

Evolution of Instruction Fine-tuning

- More strengthen instruction fine-tuning over time
  - Chain-of–thought: Generates intermediate reasoning steps in natural language before producing the final answer.

    Question: If a book costs $12 and I buy 3 books, how much do I pay?
    Reasoning: Each book costs $12. Buying 3 books means 12 × 3 = 36.
    Answer: $36

  - Step-by-step reasoning: Solves a problem by explicitly decomposing it into sequential steps.

    Question: Sort these numbers: 7, 2, 5
    Step 1: Compare the numbers.
    Step 2: The smallest is 2.
    Step 3: Then 5.
    Step 4: Then 7.
    Answer: 2, 5, 7

Evolution of Instruction Fine-tuning

- More strengthen instruction fine-tuning over time
  - Self-reflection: Revisits and verifies its own initial answer to detect and correct mistakes.

    Question: Is the answer 15?
    Initial Answer: Yes, because 10 + 5 = 15.
    Reflection: Check the calculation again. 10 + 5 is indeed 15.
    Final Answer: Yes.

  - Tool-use reasoning: Uses external tools such as calculators, search engines, or code executors during reasoning.

    Question: What is 237 × 48?
    Plan: Use a calculator because exact arithmetic is needed.
    Tool Call: calculator(237 * 48)
    Tool Result: 11376
    Answer: 11,376

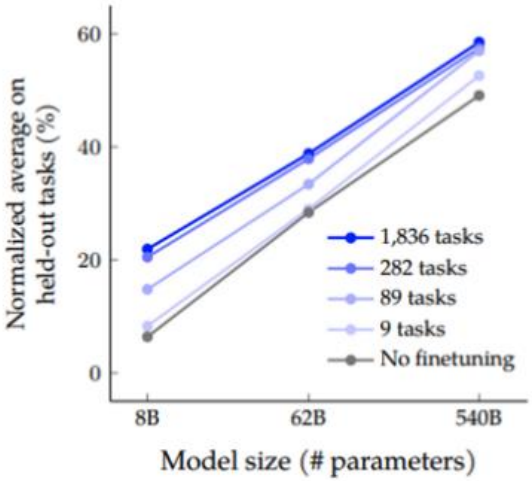
Evolution of Instruction Fine-tuning

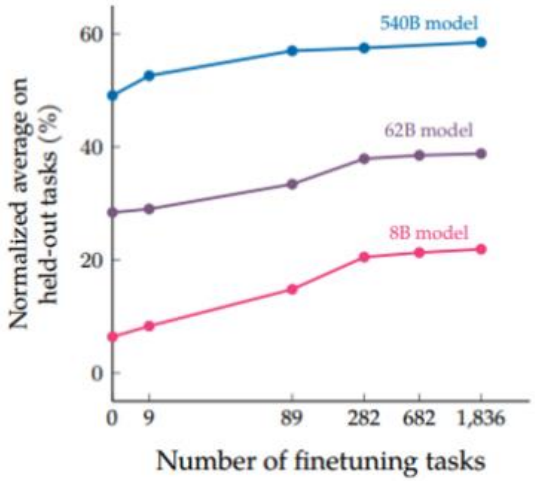
- More strengthen instruction fine-tuning over time
  - Planning trajectory: Constructs an explicit action plan before solving a complex task.

Question: Write a short summary of a paper.
Plan:
1. Read the abstract and introduction.
2. Identify the main problem.
3. Extract the proposed method.
4. Summarize the key result.
Answer: The paper addresses X by proposing Y, showing improved performance on Z.

Modern LLMs often process reasoning traces such as reasoning steps, reflections, and planning trajectories within special <think> ... </think> blocks.

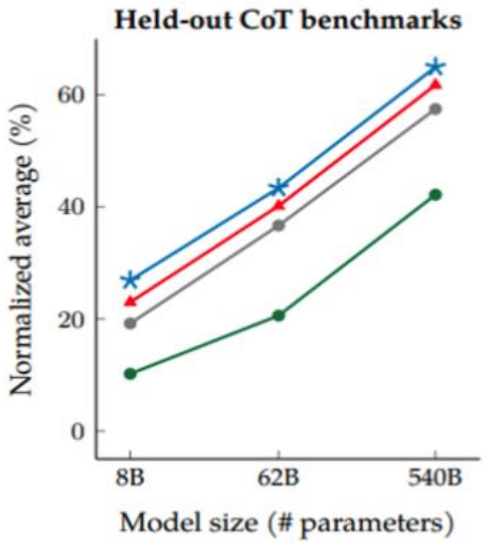
Improvement by Instruction Fine-tuning

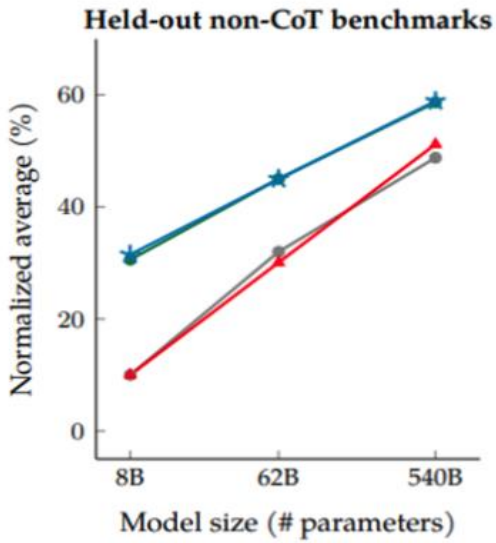


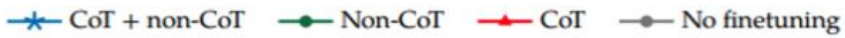


Addition of more tasks (instructions) gradually improves the model performance

Scaling model size is more beneficial







Using CoT (or other types of reasoning) comes with more improvements

KAIST

Training Objective Function for Instruction Fine-tuning

- Instruction fine-tuning optimizes the model to generate both reasoning traces and final responses conditioned on the user instruction

$$\mathcal{D} = \{(x_i, r_i, y_i)\}_{i=1}^{N}$$

x_i: instruction
r_i: reasoning/thinking tokens
y_i: final response

- The model autoregressively predicts the combined sequence:

$$z_i = [\, r_i ; y_i \,]$$ <- Output tokens

- The training objective:

$$\mathcal{L}_{\text{IFT}} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{|z_i|} \log P(z_{i,t} \mid x_i, z_{i,<t}; \theta)$$

where:

- θ: model parameters
- $z_{i,<t}$: previously generated reasoning/response tokens

In modern reasoning LLMs, r_i may include:

- Chain-of-Thought reasoning,
- self-reflection,
- planning trajectories,
- and tool-use reasoning within `<think> ... </think>` blocks.

Limitation even after Instruction Fine-tuning

- **Flexibility in interpreting instruction:** Still lacks flexibility when people express requests in various way

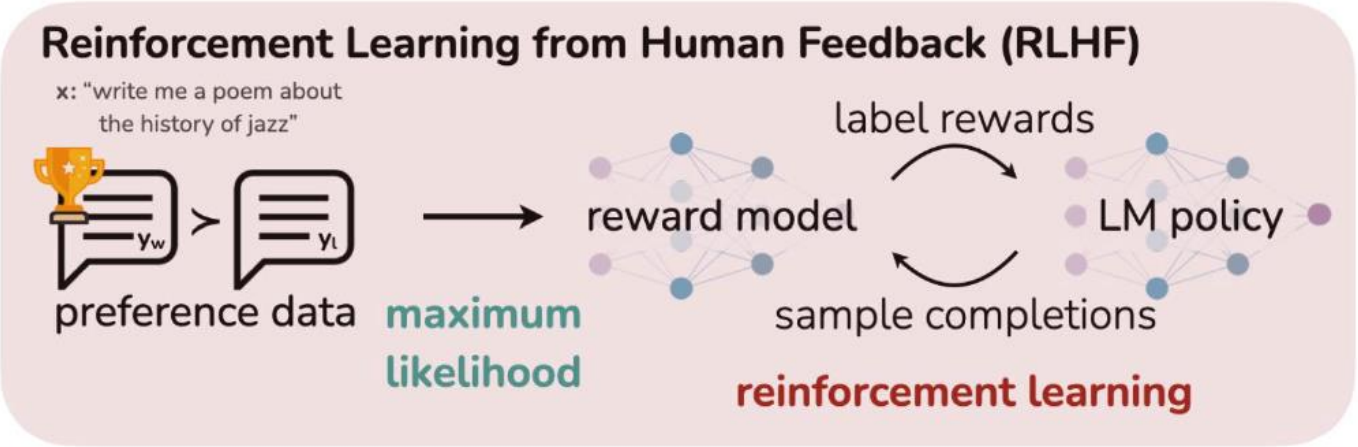
- **Safety and ethical bias:** LLMs may generate responses on sensitive topics or potentially harmful answer
  - e.g., "Hey, Let me know how to make a bomb?"

- **Missing human preference:** correctness is not the only metric to recognize real human preference
  - LLMs have to know what humans really like among candidates (output style, personal interest, etc)

This is known as **the problem of human–AI misalignment**. Humans expect AI systems to be flexible, safe, ethical, and capable of understanding diverse human preferences, but current AI systems often fail to fully satisfy these expectations.

Alignment Tuning with RLHF

- RLHF is reinforcement learning with human feedback, which trains language models to generate responses that better align with human preferences, values, and expectations



Objective:
increasing the probability of generating human's chosen responses more than human's rejected responses

Alignment Tuning with RLHF

- Core Components in RLHF:
  - **Response Pairs:** Human annotators compare multiple model responses and select the preferred one
  - **Reward Model:** Learns to predict human preference scores from the comparison data
  - **Optimization:** Reinforcement learning updates the model to maximize the reward score

Problem Formulation

$$\pi^* = \arg\max_{\pi_\theta} \mathbb{E}_{y \sim \pi_\theta(\cdot|x)} [r^*(x, y)]$$

Policy Model

Reward r^* on (prompt x, response y)

KAIST

Representative Method: PPO

- PPO refers to proximal policy optimization, an optimization process:
  - Step 1. training a reward model **r** using pair-wise comparison feedback
  - Step 2. generating a response from a given input **x**: <instruction, context>
  - Step 3. estimating the reward of the LLM's response; **r(x, y);** also value and advantage
  - Step 4. Optimizing the policy model

$$\mathcal{L}_{\text{RLHF}} = -\mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[r_\phi(x,y)\right] - \beta\,\mathbb{D}_{\text{KL}}\left[\pi_\theta(y|x)||\pi_{\text{ref}}(y|x)\right]$$

Policy Model

Reward Model

Maximizing the policy models' reward over training data

Regularizing the model being trained do not far away from the initial model

Why?

To keep the knowledge it learned from instruction fine-tuning

Limitation of PPO

- It requires many models.(policy, reference, reward), demanding high VRAM
- It incurs excessive computation due to auto-regressive decoding at each iteration of training
- It relies heavily on the imperfect (erroneous) reward model, increasing risk of bias, misalignment, and complexity – reward hacking

- In 2023, Direct Preference Optimization (DPO) was introduced at NeurIPS
  - Removes the need of a reward model
  - Only needs two models: main and reference models

DPO: Direct Preference Optimization

- Unlike PPO of two stage training, DPO needs only a single stage of training
- Objective minimization:

Reward Maximization

Reference Model Consistency

PPO Objective

$$\mathcal{L}_{\text{RLHF}} = -\mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[r_\phi(x,y)\right] - \beta\,\mathbb{D}_{\text{KL}}\left[\pi_\theta(y|x)||\pi_{\text{ref}}(y|x)\right]$$

DPO Objective

$$\mathcal{L}_{\text{DPO}}(\pi_\theta;\pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w\mid x)}{\pi_{\text{ref}}(y_w\mid x)} - \beta\log\frac{\pi_\theta(y_l\mid x)}{\pi_{\text{ref}}(y_l\mid x)}\right)\right]$$

"Consistency Term"

(Target-Reference model's relative ration -> 1.0)

"Preference Term"

When A > B,
A generation prob goes up
B generation prob goes down

KAIST

Another Solution over PPO: GRPO

- GRPO is a reinforcement learning method that improves language models by optimizing responses based on their **relative ranking within a sampled group**, without requiring a separate value model
  - PPO, DPO only use pair-wise data (A > B)
  - But GRPO can use the relative ranking, e.g., A > B > C > D > E

$$\underset{\theta}{\text{maximize}}\, \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \hat{A}_t - \beta\, \text{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_\theta(\cdot \mid s_t)] \right]$$

Policy Ratio

Advantage

Reward - Average reward

$A_t = r^*(x_t, y_t) - V(x_t)$ (advantage)

$V(x_t) = \mathbb{E}_{y \sim \pi_\theta(\cdot|x_t)}[r^*(x_t, y)]$

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t) \right]$$

Clipping to prevent too much update

- GRPO is an improved version of PPO for preference optimization
  - No separately trained reward model → single-stage training
  - No requirement for pairwise preference feedback → on-policy RL
  - Group-based comparison of self-generated outputs → relative rewarding

Another Solution over PPO: GRPO

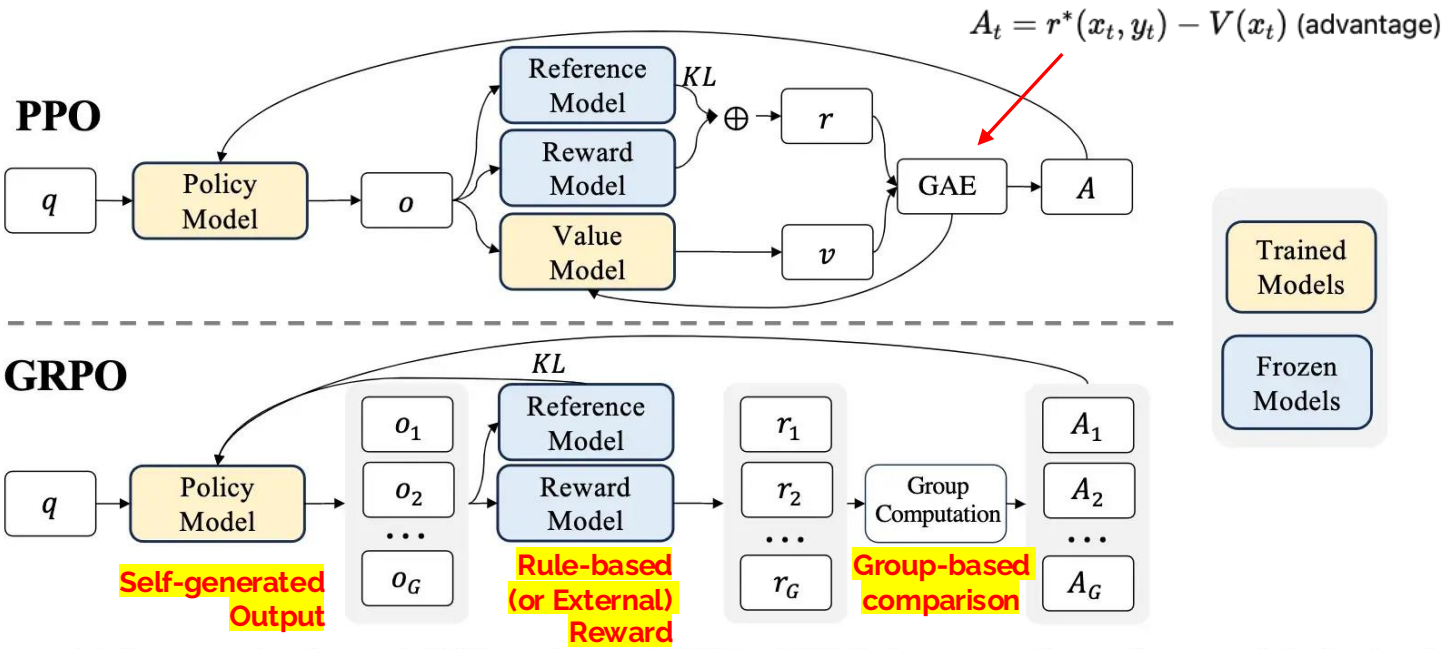


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

31

Comparison: PPO, DPO, and GRPO

- There are four distinctions over the methods
  - Is the feedback data required explicitly?
  - What type of feedback is required to optimize the LLM?
  - How to define the advantage to reward the model?
  - Which reward model should be utilized?

Distinction	PPO	DPO	GRPO
Feedback Data	Pairwise Data (for Reward Model)	Pairwise Data (for Policy Model)	Not needed (Single stage on-policy)
Feedback Composition	Scalar Reward	Pairwise Preference	Group-level Preference (Relative Reward)
Advantage Signal	Reward - Value	Preference margin (Chosen – Rejected)	Standardization over Group
Reward Model	Typically Required	Not needed	Rule-based Metric or External Reward

32

KAIST

Other Solutions?

- There have been a lot of seminar studies to improve the alignment tuning process toward a stronger LLMs

- **SimPO [NeurIPS 2024]:**
  - Removal of the reference model
  - Instead, control the preference margin
  - Additionally, length normalization

- **KTO [ICML 2025]:**
  - Do not use pairwise feedback, just using binary desirability labels (Good or Bad)
  - Allows scaling with cheaper, more abundant binary feedback data.

- **Others:** **beta-DPO** [NeurIPS 2024], **Filtered DPO** [EMNLP 2024], **Self-play** [ICLR 2025], **Diverse Preference Optimization** [ICLR 2025], etc

33

Summary

- Today, we studied advanced optimization methods over multi-stage towards foundation models
  - Pre-training to enhance the ability of generating next tokens
  - Post-training to improve the instruction-following ability and mitigate the alignment gap between human's expectation and model's actual behavior

- We skip the implementation of advanced optimization as we have a separate practice session with our TAs

34

Q & A